

vLLM 核心技术 PagedAttention 原理详解

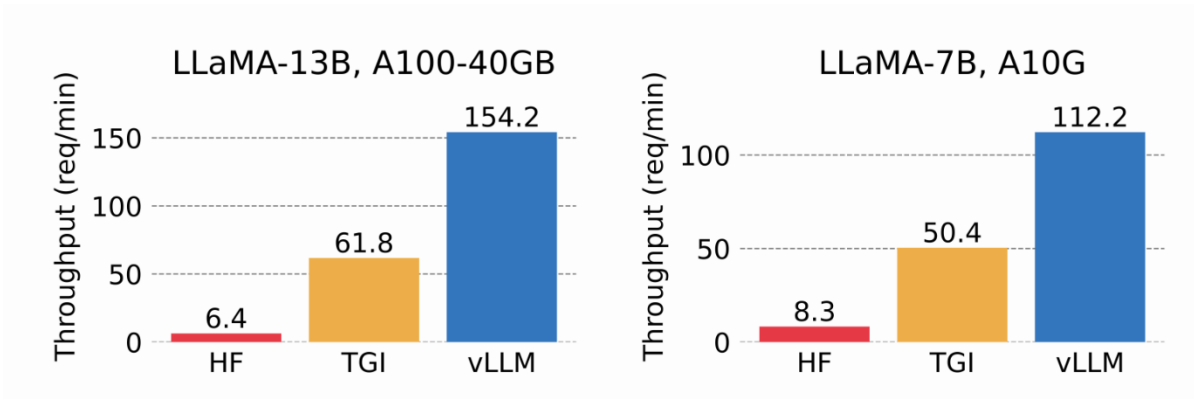
1 引言：为什么大模型推理的内存管理如此关键？

随着大语言模型（LLM）在聊天机器人、代码补全、智能问答等场景中的广泛应用，越来越多的公司开始将其作为核心服务进行部署。但运行这类模型的成本极高。相比传统的关键词搜索请求，处理一次 LLM 推理的代价可能高出十倍以上，而背后的主要成本之一，正是 GPU 内存的使用效率。

在大多数主流 LLM 中，推理过程需要缓存每一步生成过程中的 Key 和 Value 向量（即 KV Cache），以便后续生成阶段引用。这部分缓存并不会随模型权重一起常驻 GPU，而是随着用户请求的长度动态增长和释放。在高并发场景下，不合理的 KV Cache 管理方式会导致大量内存碎片和资源浪费，最终限制可并发处理的请求数量，拉低整体吞吐量。

为了解决这一瓶颈，vLLM 引入了一个全新的注意力机制——**PagedAttention**。它借鉴了操作系统中的虚拟内存分页技术，将 KV Cache 分块存储在非连续的内存地址中，配合 block-level 的共享与 copy-on-write 机制，极大提升了内存利用率，从而显著提高了模型的吞吐能力。

vLLM 团队将 vLLM 的推理吞吐量与 [HuggingFace Transformers \(HF\)](#) 和 [HuggingFace Text Generation Inference \(TGI\)](#) 进行了对比。评估在两种硬件设置下进行：在 NVIDIA A10G GPU 上运行 LLaMA-7B 模型，以及在 NVIDIA A100 (40GB) GPU 上运行 LLaMA-13B 模型。实验结果表明，与 HF 相比，vLLM 的吞吐量最高可达 24 倍，与 TGI 相比，吞吐量最高可达 3.5 倍。

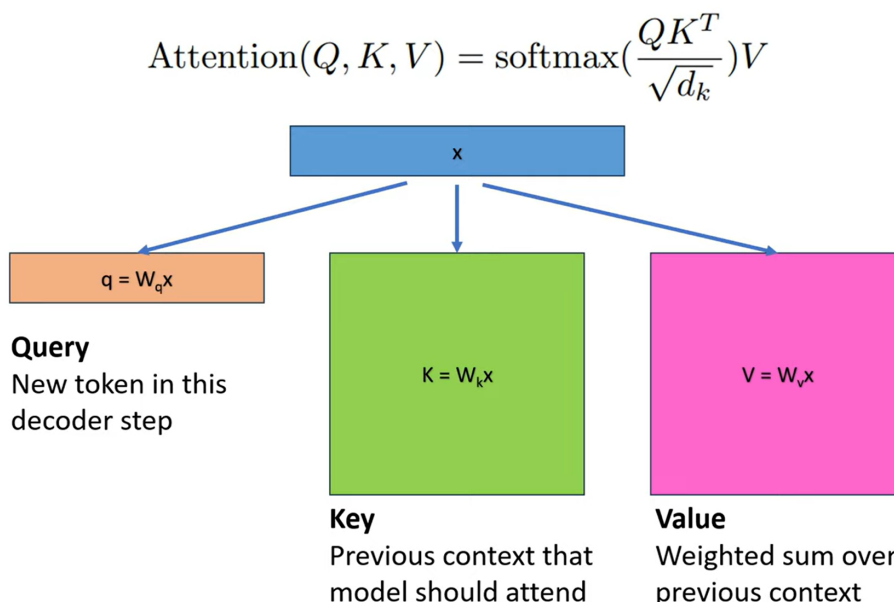


本文将结合 PagedAttention 的论文《[Efficient Memory Management for Large Language Model Serving with PagedAttention](#)》，深入解析 PagedAttention 的设计理念与实现细节，并说明它是如何有效缓解内存瓶颈，显著提升大模型推理性能的。

2 背景知识：LLM 推理中 KV Cache 的角色

在大语言模型（LLM）如 GPT、OPT、LLaMA 的推理过程中，一个关键机制是**自回归生成（autoregressive generation）**。这意味着模型会基于用户提供的 prompt（提示词），逐步生成下一个 token，每一步都依赖之前生成的 token。这种生成方式的效率，极大依赖于 Transformer 架构中的**自注意力机制（self-attention）**。

在 self-attention 中，模型为每个 token 计算三个向量：**Query (Q)**、**Key (K)** 和 **Value (V)**。每生成一个新 token，模型会将当前的 Query 向量与之前所有 token 的 Key 向量进行点积计算注意力分数，再据此对 Value 向量做加权求和。这种计算需要频繁访问此前的 token 信息。



为了避免重复计算这些历史 Key 和 Value 向量，推理系统通常会将它们缓存下来，称为 **KV Cache (Key-Value 缓存)**。这不仅节省了大量重复计算，也显著提升了推理效率。

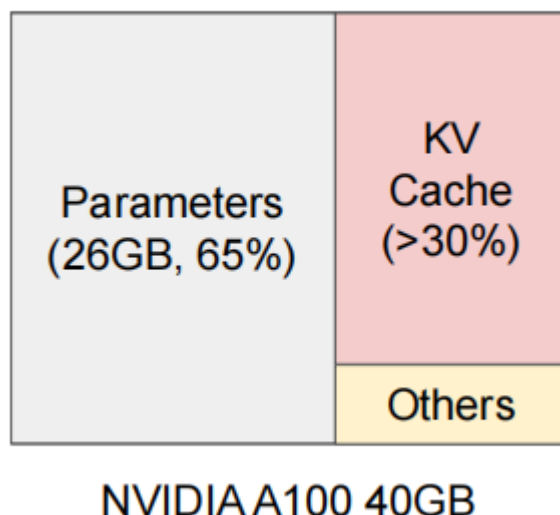
推理过程可以划分为两个阶段：

- **prefill 阶段**：模型接收完整的用户输入 prompt，并一次性并行计算所有 token 的 Query、Key 和 Value 向量。这一阶段是高度并行的，能够充分利用 GPU 的算力资源，因此属于 **compute-bound (计算受限)**，瓶颈在于算力而非内存。
- **decode 阶段**：此阶段模型开始逐个 token 地生成输出。每一步仅处理一个新的 token，模型首先计算其对应的 Query、Key 和 Value 向量。当前 token 的 Query 会与历史及当前的 Key 向量进行点积计算，通过 softmax 得到 attention 权重后，再与历史及当前的 Value 向量进行加权求和，得到当前 token 的 attention 输出。随后，当前步骤产生的 Key 和 Value 向量会被追加写入 KV Cache，以供后续使用。由于该过程是串行的、难以并行加速，且频繁读写缓存数据，因此整体表现为 **memory-bound (内存受限)**，瓶颈主要在 KV Cache 的存储与访问效率。

可以看出，prefill 阶段更侧重于并行计算，而 decode 阶段的性能很大程度上取决于 KV Cache 的内存管理能力。随着生成的 token 数量增加，KV Cache 会线性增长，并逐步占据大量 GPU 显存。

下图展示了一个 13B 参数规模的大语言模型在 NVIDIA A100 40GB 显存 GPU 上推理时的显存使用分布。其中：

- **Parameters (26GB, 65%)**：指模型的权重参数，这部分在加载模型后常驻显存，大小固定；
- **KV Cache (>30%)**：用于存储每个请求的历史 Key 和 Value 向量，随着生成 token 数量动态增长，是最主要的动态内存开销来源；
- **Others**：主要指推理过程中暂时产生的中间计算结果（如 activation），生命周期短，通常在计算完一层后即被释放，占用较少。



从这张图可以直观看出，KV Cache 是模型推理过程中最主要的“动态”显存负担，它不仅大，而且会随着请求数量和序列长度迅速膨胀。因此，如何高效管理 KV Cache，决定了一个推理系统能否支撑大批量并发请求，是提高吞吐量的关键。

3 内存管理的挑战：KV Cache 为什么成了 LLM 推理的瓶颈？

当前主流的 LLM 推理系统在 KV Cache 的内存管理上普遍存在 3 类结构性问题：显存占用增长快、内存碎片严重，以及缓存难以复用。

3.1 KV Cache 占用迅速增长，极易耗尽显存

以 OPT-13B 为例，论文指出其每个 token 的 KV Cache 占用约为 **800KB**，计算方式为：

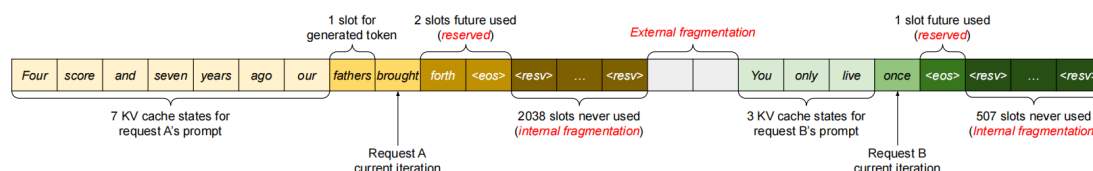
$$2 \text{ (Key 和 Value)} \times 5120 \text{ (hidden size)} \times 40 \text{ (Transformer 层数)} \times 2 \text{ 字节 (FP16)} \\ \approx 819,200 \text{ 字节} \approx 800\text{KB}$$

如果生成完整的 2048 个 token，单个请求的 KV Cache 占用就可达约 1.6GB。在一块 40GB 显存的 A100 GPU 上，这种增长速度意味着仅同时处理少量请求，就可能达到内存瓶颈，直接限制了批处理规模和系统吞吐量。

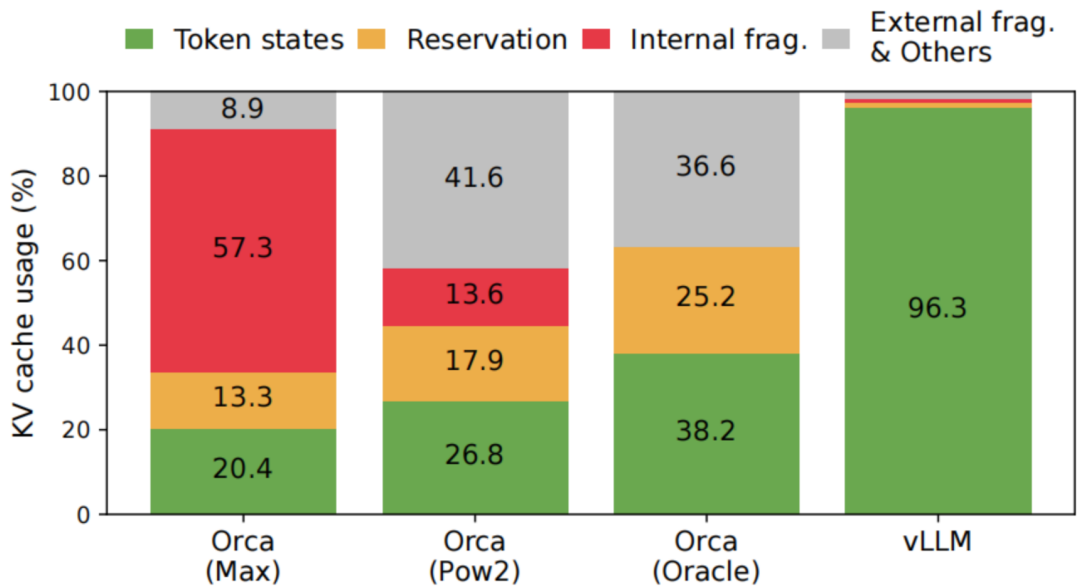
3.2 预分配导致内存碎片严重

传统推理系统（如 FasterTransformer 和 Orca）通常采用**预分配**策略：请求开始时，就按最大可能生成的长度（如 2048 token）为每个请求分配一整块连续内存空间。这种方式带来 3 类显著的内存浪费：

- **已预留但尚未使用的空间 (Reserved)**：虽然未来会使用，但当前暂未使用到。
- **内部碎片 (Internal Fragmentation)**：实际 token 数小于预留长度，剩余空间浪费。
- **外部碎片 (External Fragmentation)**：不同请求所需内存大小不一，造成内存块间不连续，产生碎片。



论文中的实验结果显示，再传统系统中真正用于存放 KV Cache 的有效内存占比最低仅约 20.4%，其余全为浪费。

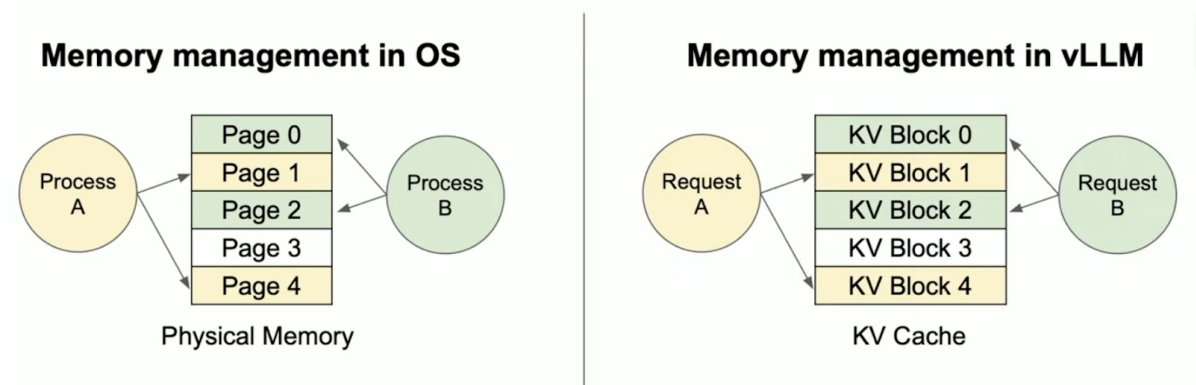


3.3 KV Cache 难以共享，内存复用受限

在 **Parallel Sampling** 或 **Beam Search** 等复杂推理模式中，一个请求可能包含多个生成分支（如多个候选答案），这些分支共享相同的 prompt，因此理论上可以共享其 KV Cache。但传统系统将每个分支的 KV Cache 存放在独立的连续内存中，物理上无法实现共享，只能进行冗余复制，进而显著增加了内存开销。

4 核心创新：PagedAttention 是什么？

为了解决 KV Cache 内存管理中的高占用、严重碎片和复用困难等问题，vLLM 提出了一种全新的注意力机制 —— **PagedAttention**。它的核心思想，借鉴自操作系统中广泛应用的**虚拟内存分页机制 (Virtual Memory & Paging)**。



在操作系统中，虚拟内存将程序的地址空间划分为固定大小的“页”（Page），这些页可以映射到物理内存中任意位置，实现非连续分配，从而有效解决了内存碎片和共享问题。PagedAttention 将这一思路引入 KV Cache 的管理中，并带来了 3 项关键改进：

1. **KV Cache 被切分为固定大小的 block**：PagedAttention 将每个序列的 KV Cache 切分为固定大小的 block（默认是 16 个 token），每个 block 存储若干个 token 的 Key 和 Value 向量。这种设计统一了内存分配粒度，使系统能够以更标准化的方式管理 KV Cache 的分配与回收，从而提升内存复用效率，并有效减少内存碎片。

2. **block 可以存放在非连续的物理内存中**：与传统的 Attention 不同，PagedAttention 不再要求这些 KV 向量在内存中连续排列，而是通过逻辑 block 与物理 block 的映射，实现非连续存储。映射关系由 block table 维护，它类似于操作系统中的页表，用于记录每个逻辑 block 对应的物理内存位置，确保模型在推理过程中可以正确读取所需的 KV Cache。
3. **支持灵活的分配与释放，以及共享机制**：PagedAttention 支持按需分配和回收 block，并允许多个序列共享 block。PagedAttention 使用了 copy-on-write (CoW) 机制，允许多个生成样本共享大部分输入 prompt 的 KV Cache，只有当某个分支需要写入新 token 的 KV 数据时，系统才会将相关 block 复制到新的物理位置，从而在保证数据隔离的同时极大地节省显存资源，提升推理效率与吞吐量。

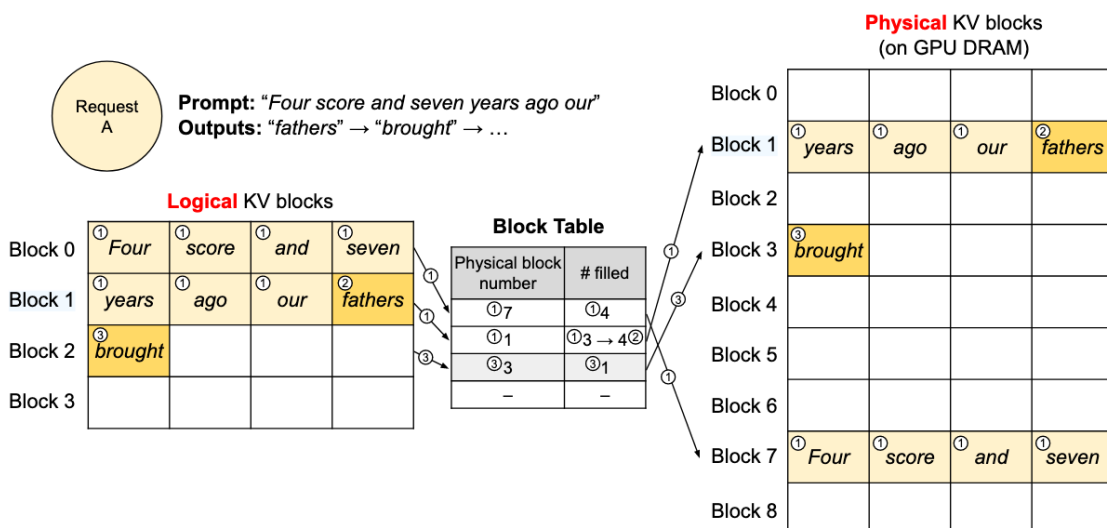
5 实现细节：vLLM 如何利用 PagedAttention 实现高效推理

5.1 推理过程中的内存管理

vLLM 在 decode 阶段使用 PagedAttention 配合 block-level 的内存管理机制，实现了高效、动态的 KV Cache 管理。

如下图所示，用户请求的 prompt 为 "Four score and seven years ago our"，共包含 7 个 token：

1. **prefill 阶段**：vLLM 为前 7 个 token 分配两个逻辑块 block 0 和 block 1，分别映射到物理块 7 和 1。block 0 存储前 4 个 token，block 1 存储后 3 个 token 及第一个生成 token "fathers"，填充数为 4。
2. **decode 阶段 - 生成第 1 个词**：生成 token "brought"，由于 block 1 尚未填满（最多容纳 4 个 token），因此直接将新 KV Cache 写入该块，填充计数从 3 更新为 4。
3. **decode 阶段 - 生成第 2 个词**：生成下一个 token，此时 block 1 已满，系统为逻辑块 block 2 分配新的物理块 block 3，并写入 KV Cache，同时更新映射表。



整个过程中，每个逻辑块仅在前一个块被填满后才会分配新的物理块，从而最大程度地减少内存浪费。不难发现，在计算时我们操作的是逻辑块，也就是说，这些 token 在形式上是连续排列的；与此同时，vLLM 会通过 block table 映射关系，在后台将逻辑块映射到实际的物理块，从而完成数据的读取与计算。通过这种方式，每个请求仿佛都在一个连续且充足的内存空间中运行，尽管这些数据在物理内存中实际上是非连续存储的。

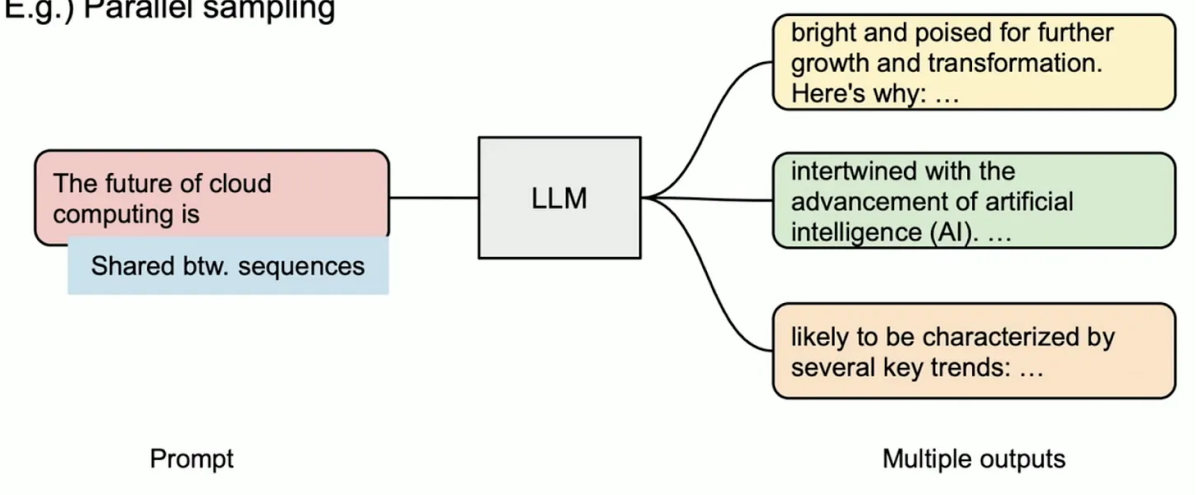
5.2 支持多样化的推理策略

PagedAttention 中采用的基于分页的 KV Cache 管理机制，不仅在常规单序列生成中表现出色，也天然适合多种复杂的解码策略。

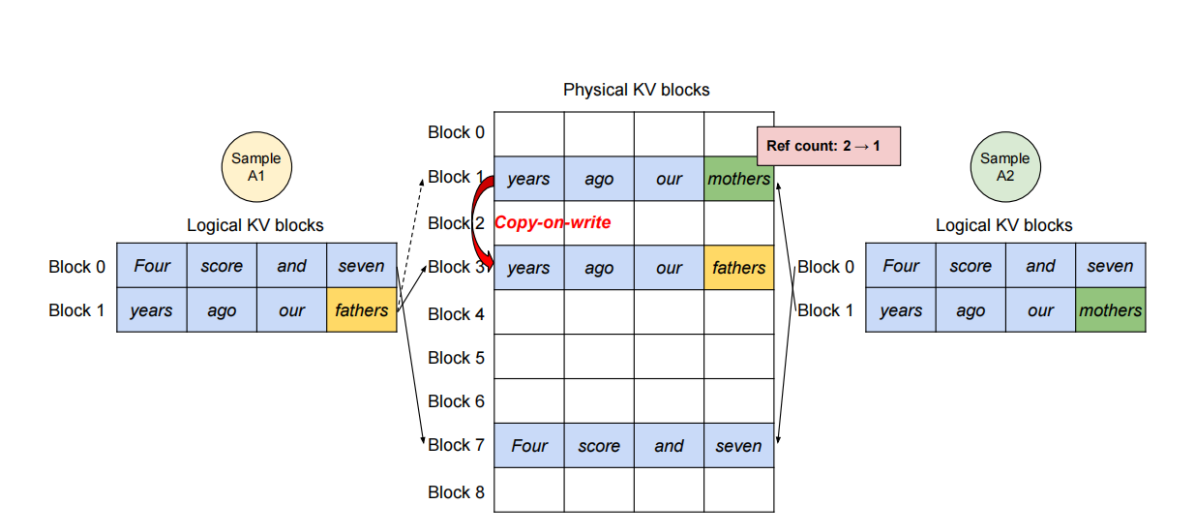
5.2.1 并行采样 (Parallel Sampling)

在 Parallel Sampling 中，同一个 prompt 会生成多个候选输出，便于用户从多个备选中选择最佳响应，常用于内容生成或模型对比测试。

E.g.) Parallel sampling

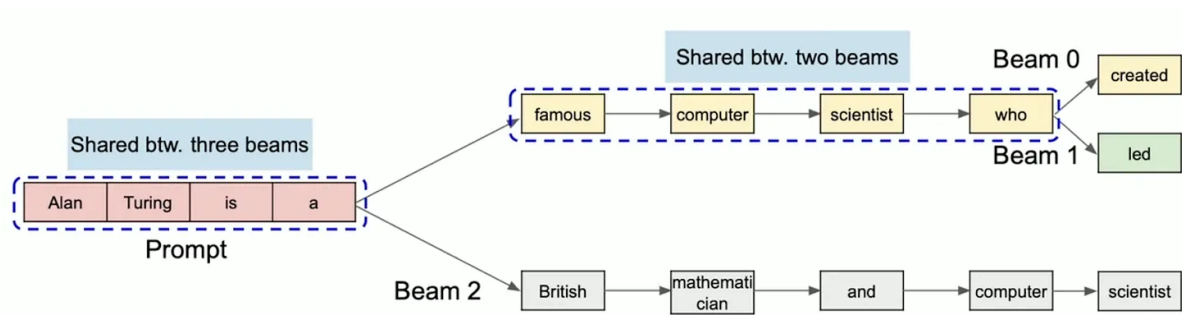


在 vLLM 中，这些采样序列共享相同的 prompt，其对应的 KV Cache 也可以共用同一组物理块。PagedAttention 通过引用计数和 block-level 的 copy-on-write 机制实现共享与隔离的平衡：只有当序列出现不同分支时，才会触发复制操作。

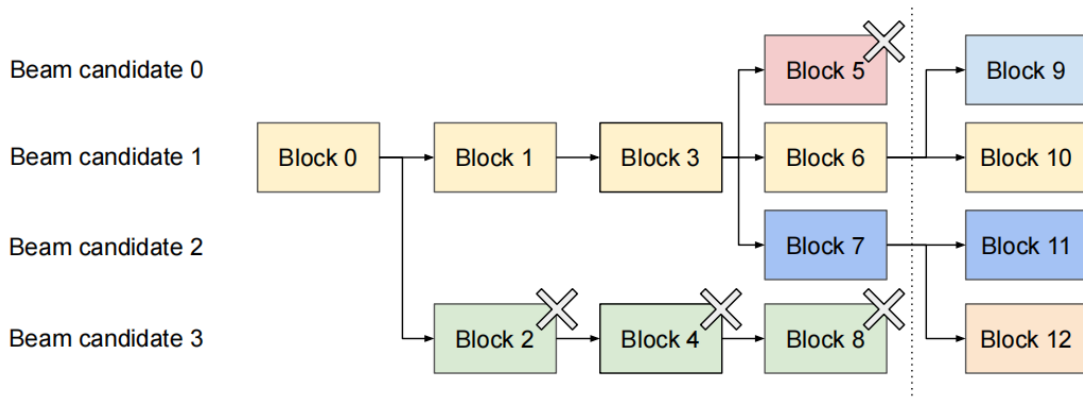


5.2.2 束搜索 (Beam Search)

Beam Search 是机器翻译等任务中常见的解码策略。它会维护多个“beam”路径，每轮扩展最优候选并保留 top-k 序列。

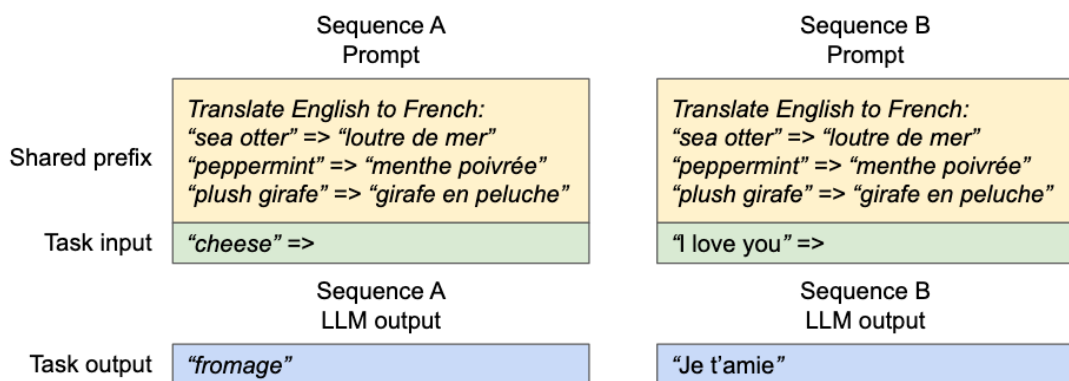


在 vLLM 中，多个 beam 可以共享公共前缀部分的 KV Cache，不仅包括输入 prompt，还包括生成过程中的公共前缀 token。只要这些 beam 的生成路径尚未分叉，它们就会复用相同的物理块。当路径分叉发生后，vLLM 才通过 copy-on-write 机制对共享块进行拆分，从而保证每个 beam 的独立性。



5.2.3 共享前缀 (Shared Prefix)

在许多提示工程实践中，多个请求会以相同的系统提示或 few-shot 示例开头（例如翻译任务中的多个例句）。这些共享前缀同样可以被 vLLM 缓存并复用。



5.2.4 混合解码 (Mixed Decoding)

前面提到的几种解码方式（如 Parallel Sampling、Beam Search、Shared Prefix 等）在内存的共享和访问方式上存在差异，传统系统很难同时高效地处理这些不同策略的请求。而 vLLM 则通过一个通用的映射层（block table）屏蔽了这些差异，让系统能够在统一框架下处理多样化的请求。

5.3 推理任务的调度与抢占

当请求量超过系统处理能力时，vLLM 需要在有限的显存资源下合理调度推理请求，并对部分请求进行抢占（Preemption）。为此，vLLM 采用了 **FCFS (first-come-first-serve)** 的策略，确保请求被公平处理：优先服务最早到达的请求，优先抢占最近到达的请求，避免资源饥饿。

由于大模型的输入 prompt 长度差异大，输出长度也无法预估，随着请求和生成的 token 增加，GPU 中用于存储 KV Cache 的物理块可能耗尽。此时，vLLM 面临两个核心问题：

1. 应该回收哪些块？
2. 如果后续仍需使用，被回收的块如何恢复？

5.3.1 块回收策略：All-or-Nothing

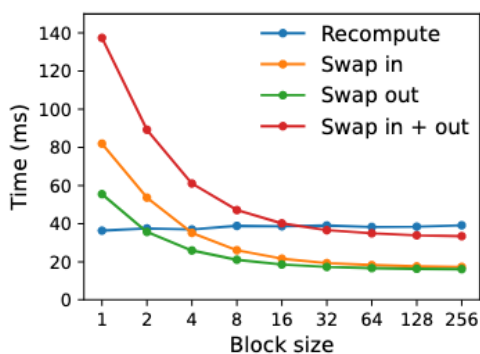
vLLM 中每个序列的所有 KV Cache 块始终一起被访问，因此采用 “All-or-Nothing” 的回收策略：要么完全回收一个请求的全部 KV Cache，要么不回收。此外，像 Beam Search 这类单请求中含多个子序列（beam）的情况，这些序列之间可能共享 KV Cache，必须作为一个 “sequence group” 整体抢占或恢复，确保共享关系不被破坏。

5.3.2 块恢复策略：Swapping 与 Recomputation

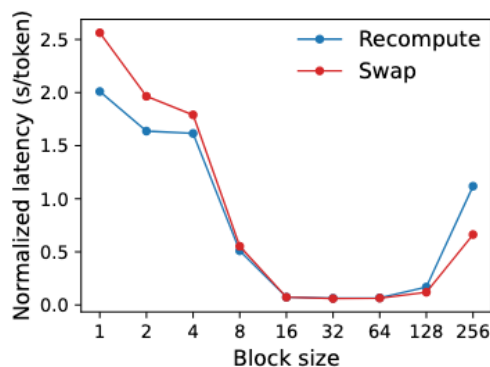
对于被抢占请求的 KV Cache，vLLM 提供两种恢复机制：

- **Swapping (交换)**：将被抢占请求的 KV Cache 块从 GPU 移动到 CPU 内存。当有空闲 GPU 块时再从 CPU 恢复回来继续执行。为了控制资源使用，被交换（swapped）到 CPU 内存中的 KV Cache 块数量，永远不会超过 GPU 中物理块的总数。
- **Recomputation (重计算)**：直接丢弃已生成的 KV Cache，待请求恢复后重新计算。Recomputation 可将已生成的 token 与原始 prompt 拼接为新的输入，一次性完成所有 KV Cache 的预填充，其开销显著低于原始 decode 阶段逐 token 生成的方式，随后可以继续后续的 decode 过程。

根据论文中的实验结果，Swapping 更适用于 block size 较大的场景，而 Recomputation 则在不同 block size 下的表现更为稳定。在中等 block size（16 到 64）范围内，两种方式的端到端性能基本相当。



(a) Microbenchmark

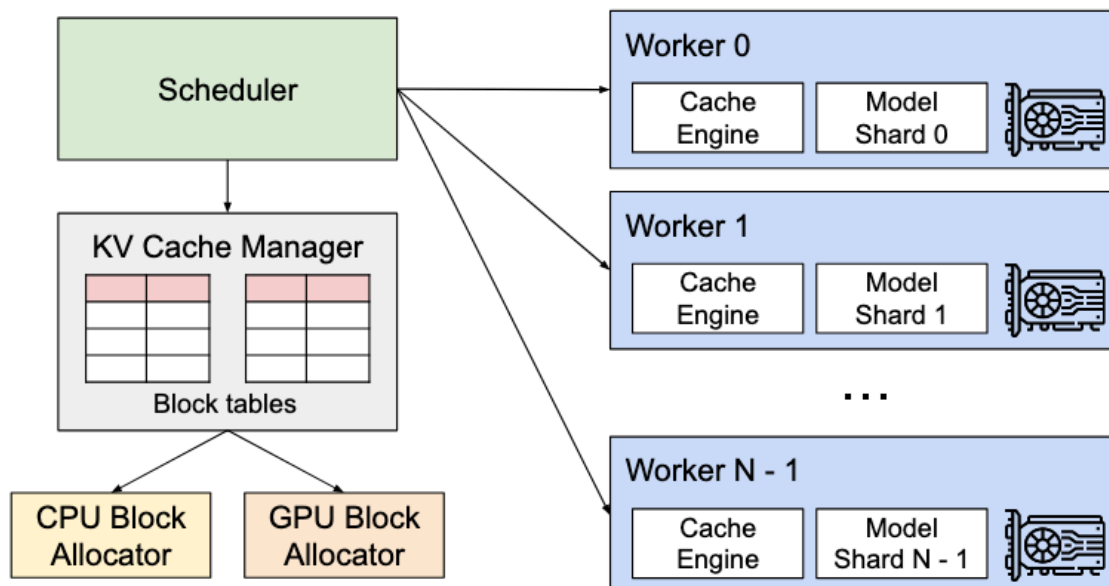


(b) End-to-end performance

5.4 分布式架构

随着大语言模型（LLM）参数规模的不断扩大，许多模型的总参数量早已超出单张 GPU 的显存上限。因此，必须将模型参数切分并分布在多张 GPU 上，以实现并行执行，这种策略被称为 **模型并行 (Model Parallelism)**。这不仅要求模型本身具备良好的并行性，也对系统提出了更高要求——需具备能够协调跨设备内存访问与管理的能力。

为应对这一挑战，vLLM 构建了一套面向分布式推理的执行架构，原生支持 [Megatron-LM](#) 风格的张量并行策略，并通过统一调度与基于分页的 KV Cache 管理机制，实现了跨 GPU 的高效协同推理与资源共享。



在每一步 decode 过程中，调度器首先为每个 batch 中的请求准备控制信息，其中包括：

- 输入 token 的 ID；
- 以及该请求的块表（block table，即逻辑块到物理块的映射信息）。

接下来，调度器会将这些控制信息广播给所有的 GPU worker。

然后，GPU worker 开始使用这些输入 token ID 执行模型推理。在注意力层中，GPU worker 会根据控制信息中提供的块表，读取对应的 KV Cache。在执行过程中，GPU worker 会通过 all-reduce 通信原语同步中间计算结果，这一过程无需调度器参与协调。最后，GPU worker 会将本轮生成的 token（采样结果）发送回调度器。

5.5 GPU kernel 优化

PagedAttention 引入了非连续、按块访问 KV Cache 的内存访问模式，而这些访问模式并不适配现有推理系统中的通用 kernel 实现。为此，vLLM 专门实现了一系列 GPU kernel 优化，以充分发挥硬件性能：

- **融合 reshape 与块写入 (Fused reshape and block write)**
在 Transformer 的每一层，新生成的 KV Cache 需要被分块、reshape 成适合按块读取的内存布局，并根据 block table 的位置写入显存。vLLM 将这些操作融合为一个 kernel 执行，避免了多次 kernel 启动的开销，从而提升执行效率。
- **融合块读取与注意力计算 (Fusing block read and attention)**
vLLM 在 FasterTransformer 的基础上改造了注意力 kernel，使其能够根据 block table 实时读取 KV Cache 并执行 attention 操作。为了确保内存访问的合并性 (coalesced access)，vLLM 为每个块分配一个 GPU warp 来读取数据，并支持同一 batch 中不同序列长度的处理，增强了执行灵活性。
- **优化块复制操作 (Fused block copy)**
copy-on-write 机制触发的块复制可能涉及多个非连续内存块。若使用 `cudaMemcpyAsync`，会造成频繁的小数据移动，效率低下。vLLM 实现了一个融合的 kernel，可将多个块复制操作批量合并为一次 kernel 启动，显著降低了调度开销与执行延迟。

6 总结

本文系统梳理了 vLLM 核心技术 PagedAttention 的设计理念与实现机制。文章从 KV Cache 在推理中的关键作用与内存管理挑战切入，介绍了 vLLM 在请求调度、分布式执行及 GPU kernel 优化等方面的核心改进。PagedAttention 通过分页机制与动态映射，有效提升了显存利用率，使 vLLM 在保持低延迟的同时显著提升了吞吐能力。

7 附录

附录中包含了在阅读 PagedAttention 的论文 [《Efficient Memory Management for Large Language Model Serving with PagedAttention》](#) 时可能需要解释的相关概念。

什么是注意力？

当模型处理句子 “an apple and an orange” 时，注意力机制帮助模型理解 “apple” 的具体含义。

- “apple” 是一个多义词，既可以指水果，也可以指品牌（Apple 公司）。
- 句子中紧跟着的 “orange” 明确是一种水果，这个上下文信息通过注意力机制被捕捉到。
- 由于 “orange” 与 “apple” 在水果的语义空间中关系更密切，注意力机制会赋予 “orange” 较高的权重，从而帮助模型倾向于将 “apple” 理解为水果，而非品牌。

什么是 Q、K、V？

在 Transformer 模型中，**Q (Query)**、**K (Key)**、**V (Value)** 是自注意力机制 (Self-Attention) 的核心组成部分，用于计算输入序列中不同 token 之间的依赖关系，并融合上下文信息。

Q、K、V 都是由输入的词向量 (embedding) 通过不同的线性变换矩阵生成的：

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

- 每个权重矩阵 W^Q, W^K, W^V 都是可训练的参数，能够增强模型的表达能力；
- 这三个向量随后被送入注意力机制中进行信息交互与聚合。

Query (Q)

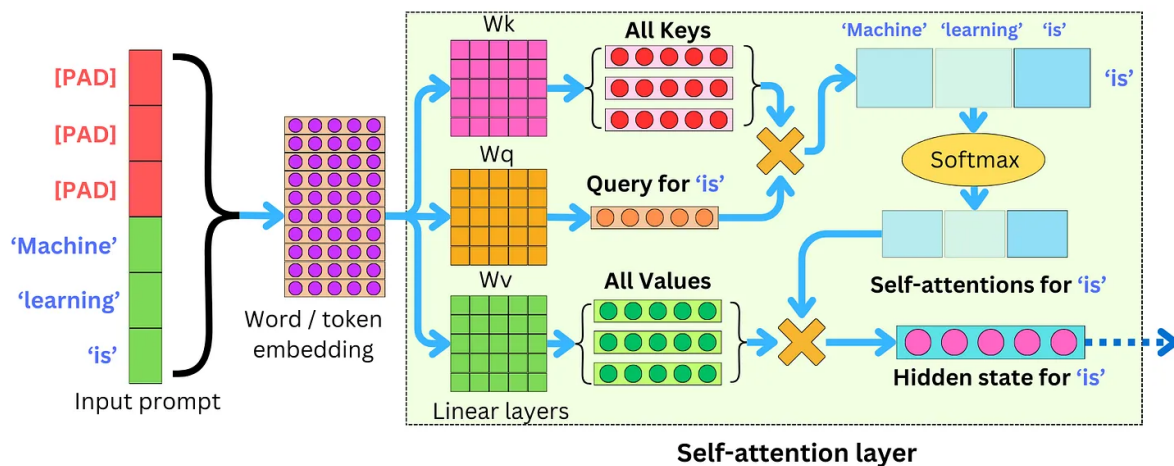
- 定义：**当前 token 提出的“问题”或“关注点”。
- 作用：**用于与所有 token 的 Key 做点积，计算相似度（相关性分数）；

Key (K)

- 定义：**所有 token 的“身份标识”或“关键词”；
- 作用：**与 Query 做点积，用于计算注意力分数，表示被关注的程度。

Value (V)

- 定义：**包含 token 的实际语义信息；
- 作用：**在 softmax 得到的注意力权重下进行加权聚合，形成上下文感知表示。



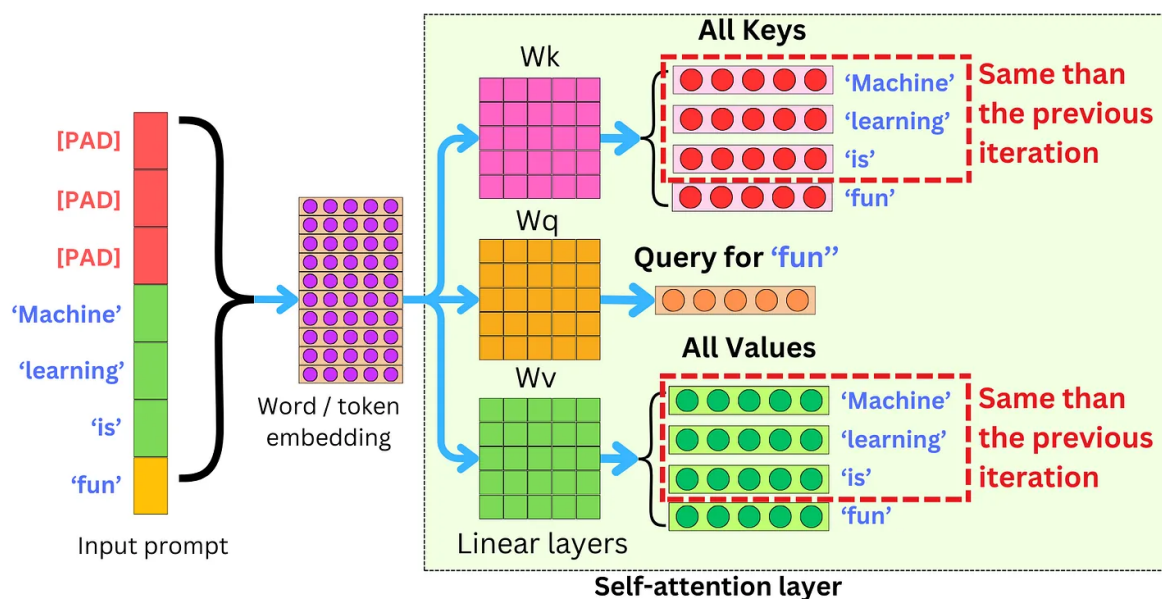
Attention 的输出会传入一个 **Linear 层** 映射成词表大小的 logits，然后经过 **softmax** 得到每个词的概率。最终生成阶段使用 greedy、sampling 或 beam search 等方式，从中选择下一个 token。

KV Cache

- 在自回归生成任务（如大语言模型推理）中，每一步生成新 token 时，输入序列会不断变长。每次经过自注意力层时，都需要重新计算所有 token 的 Key (K) 和 Value (V) 向量。
- 对于已经生成过的 token（如 "Machine", "learning", "is", "fun"），它们的 Key 和 Value 向量在每次新 token 推理时其实是完全相同的，并不会发生变化。
- 但如果不使用 kv cache，每次生成新 token 时，模型都要重复计算这些已经存在的 Key 和 Value，造成大量冗余计算，极大降低推理效率。

kv cache 的作用：

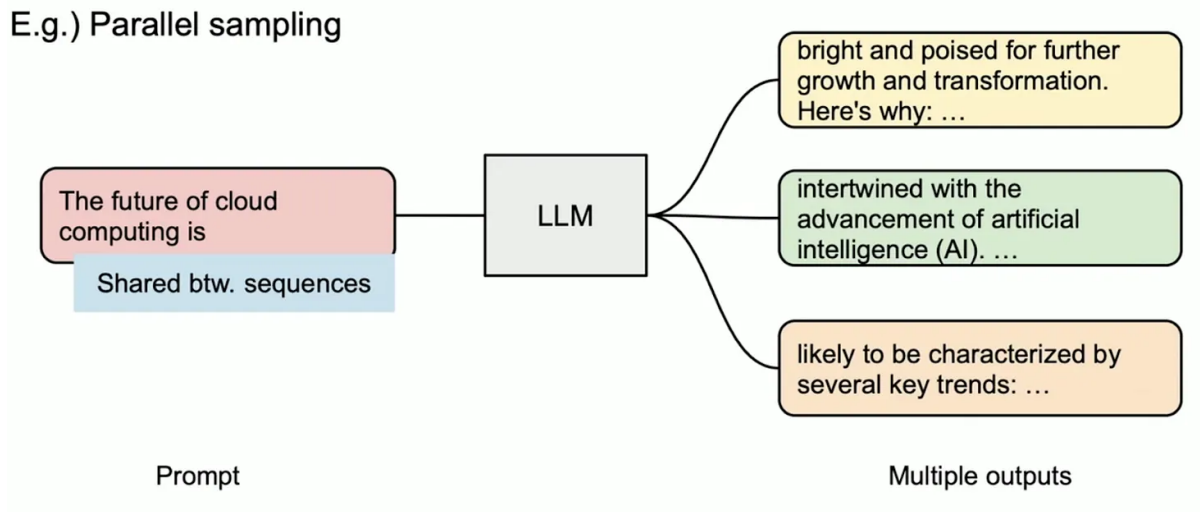
- kv cache 会把已经生成过的 token 的 Key 和 Value 向量缓存下来，后续每次推理时只需计算新 token 的 Key 和 Value，然后与缓存中的历史 Key 和 Value 拼接即可。
- 这样，模型每步只需计算一次新的 Key/Value，大幅减少重复计算，显著加快推理速度，尤其是在长序列推理时效果更明显。



Parallel Sampling

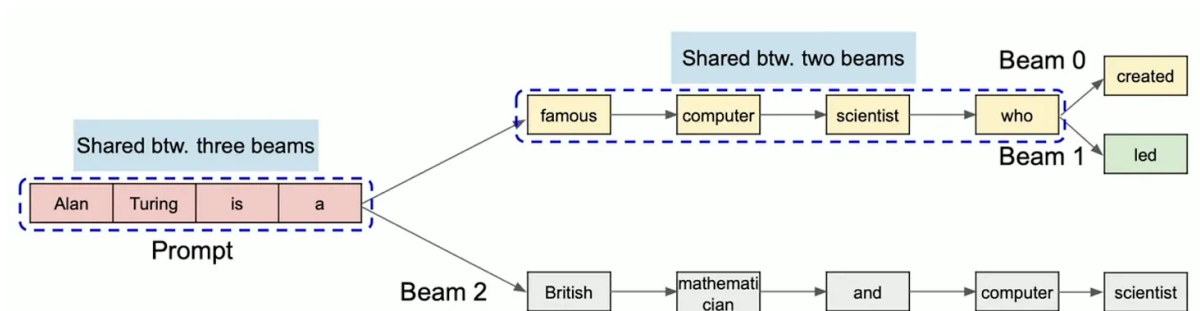
Parallel Sampling (并行采样) 指的是对同一个输入 prompt，同时并行生成多个不同的输出结果，以提升生成内容的多样性和可选性。例如，ChatGPT 可以为同一个问题生成多个答案供你挑选。

E.g.) Parallel sampling



Beam Search

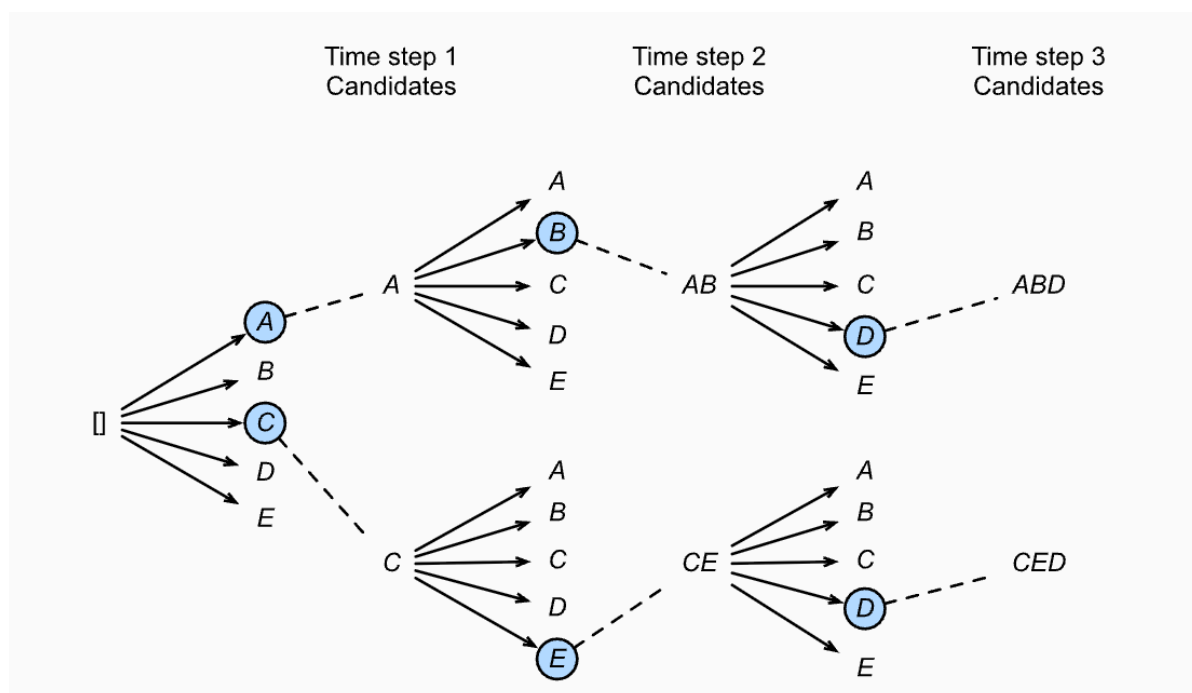
Beam Search (束搜索) 是一种在文本生成任务（如机器翻译、文本摘要、语音识别等）中常用的解码策略。它通过在每一步保留多个得分最高的候选序列，逐步扩展这些序列，以找到整体概率最高的输出。



每个时间步都有 A、B、C、D、E 五种可能的输出，设定 `num_beams = 2`，即每一步保留得分最高的两个候选序列。

1. **第一个时间步**：模型预测出 5 个候选词 A~E。假设其中 A 和 C 得分最高，因此保留 [A] 和 [C] 两个分支，舍弃其余。
2. **第二个时间步**：从 [A] 和 [C] 分别扩展出 5 个新词，形成共 10 个候选序列（如 [AB], [AD], [CE] 等）。对这 10 个序列统一打分，保留得分最高的两个（假设为 [AB] 和 [CE]）。
3. **第三个时间步**：再从 [AB] 和 [CE] 各自扩展出 5 个词，得到 10 个新候选。重复相同的筛选过程，最终保留得分最高的两个序列（如 [ABD] 和 [CED]）。

最终输出 [ABD] 和 [CED]，它们是通过 Beam Search 筛选出的两个最优结果。



虚拟内存和分页

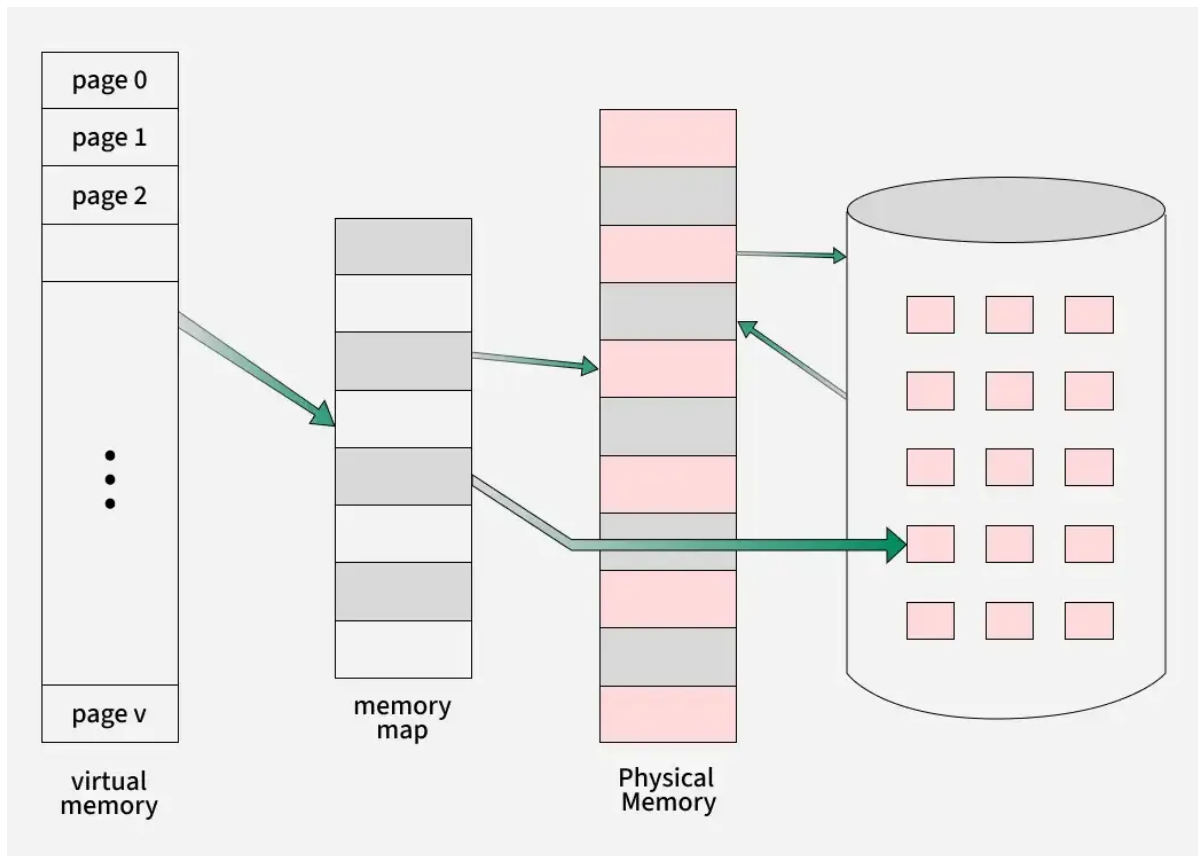
在传统的计算机系统中，CPU 直接操作物理内存地址。这意味着，应用程序认为自己是在操作物理内存。如果两个程序占用的内存有重叠，它们可能会互相干扰，导致程序崩溃。为了避免这种情况，操作系统引入了虚拟内存机制。每个进程被分配独立的一套虚拟地址空间，互不干涉。

虚拟内存 (Virtual Memory) 是计算机系统内存管理的一种技术。它使得应用程序认为它拥有连续的可用的内存（一个连续完整的地址空间），而实际上，它通常是被分隔成多个物理内存碎片，还有部分暂时存储在外部磁盘存储器上，在需要时进行数据交换。

分页 (Paging) 是一种将虚拟地址空间和物理内存划分为固定大小块的技术。在分页系统中：

- **虚拟内存**被划分为固定大小的块，称为 **页 (Pages)**。
- **物理内存**被划分为与页大小相同的块，称为 **页框 (Page Frames)**。
- **页表 (Page Table)** 是一个数据结构，用于记录虚拟页与物理页框之间的映射关系。

当程序访问某个虚拟地址时，系统会通过页表将其转换为对应的物理地址。如果所需的页不在物理内存中，就会发生 **缺页中断 (Page Fault)**，操作系统会将所需页从磁盘加载到内存中。



自回归分解 (autoregressive decomposition) 公式

语言建模的目标就是：计算一段文本（由一系列 token 构成）的概率是多少。由于语言本身具有自然的**顺序性**，所以我们通常会将这个**联合概率**分解为一系列**条件概率的乘积**，这也被称为 **自回归分解 (autoregressive decomposition)**。

$$P(x) = P(x_1) \cdot P(x_2 | x_1) \cdot \dots \cdot P(x_n | x_1, \dots, x_{n-1})$$

整个句子的概率是：第一个词的概率 × 第二个词在第一个词出现的条件下的概率 × 第三个词在前两个词出现的条件下的概率 依此类推。

Causal Self-Attention (因果自注意力) 公式

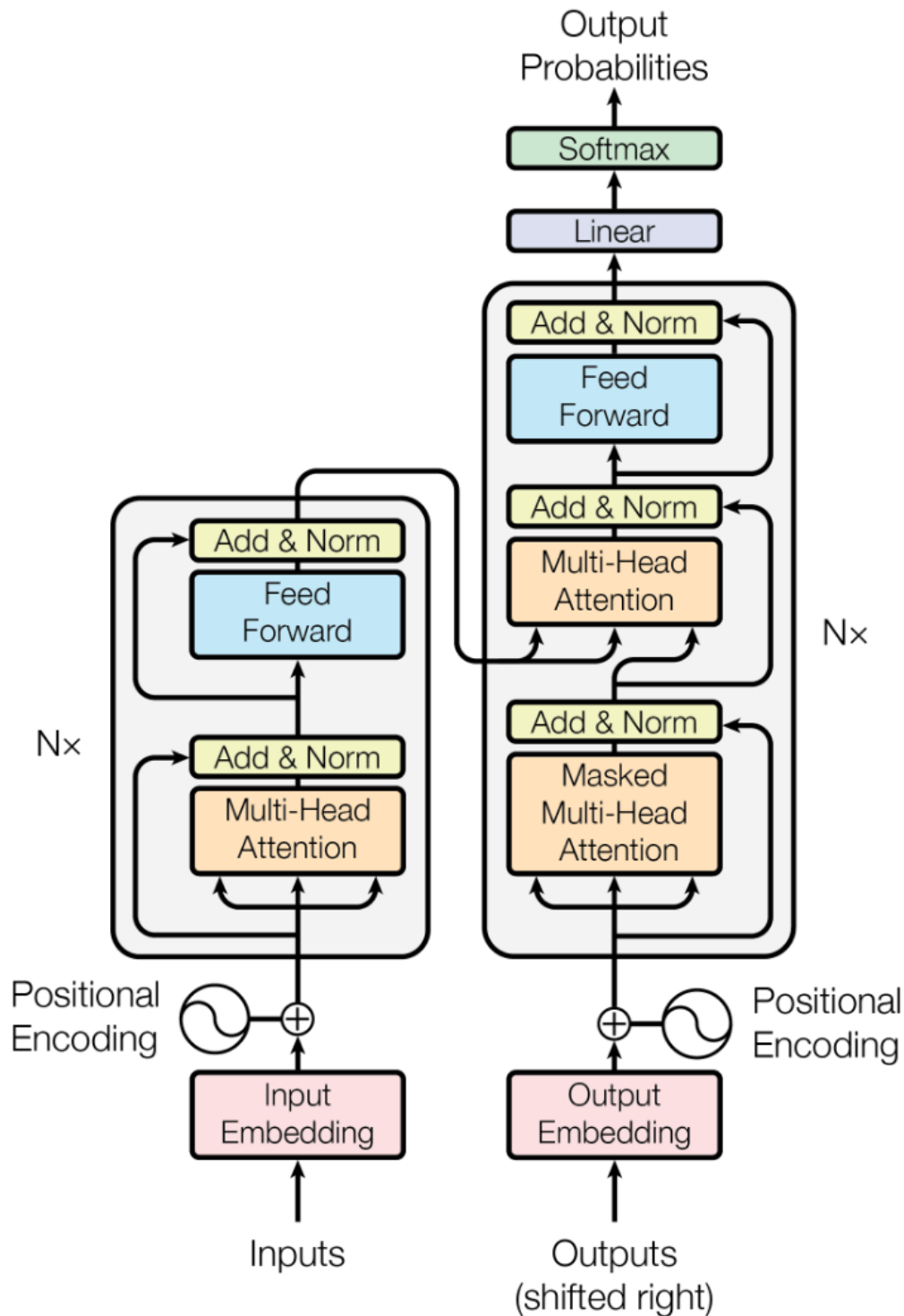
Causal Self-Attention (因果自注意力) 是一种特殊的自注意力机制，采用了因果掩码 (Causal Masking)，主要用于序列生成任务中，确保模型在生成当前位置的输出时，只依赖于该位置之前的信息，而不会利用未来位置的信息，从而保持自回归 (autoregressive) 属性。

$$a_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d})}{\sum_{t=1}^i \exp(q_i^\top k_t / \sqrt{d})}, \quad o_i = \sum_{j=1}^i a_{ij} v_j$$

- $q_i^\top k_j$: 第 i 个 token 的 Query 向量与第 j 个 token 的 Key 向量之间的点积（衡量相似度）。
- 除以 $\sqrt{d}$: 是为了防止点积值过大导致 softmax 输出太极端（归一化）。
- $\exp / \sum \exp$: 是 softmax 函数，用于将所有相似度变成一个概率分布。
- 上限是 i : 说明这里使用了**因果掩码 (causal mask)**，也就是说 token x_i 只能“看到”它自己和它之前的 tokens，不能访问未来的信息（用于生成任务）。

Transformer 架构

Transformer 架构主要有三种变体：**Encoder-Only**（仅编码器）、**Decoder-Only**（仅解码器）和 **Encoder-Decoder**（编码器-解码器）。它们在结构和应用场景上各有特点。



1. Encoder-Only（仅编码器）

- **结构特点：**仅包含编码器部分，使用双向自注意力机制（Bidirectional Self-Attention）。该机制允许模型在处理每个词时，同时关注该词前后的所有词，从而捕捉整个序列的上下文信息。
- **典型模型：**BERT（Bidirectional Encoder Representations from Transformers）
- **应用场景：**文本分类、命名实体识别、情感分析、问答系统等需要对输入文本进行理解的任务。
- **工作方式：**模型接收完整的输入序列，通过编码器生成上下文相关的表示，用于下游任务。

2. Decoder-Only (仅解码器)

- **结构特点**: 仅包含解码器部分, 采用自回归机制, 使用因果掩码 (Causal Masking) 确保每个位置只能关注其之前的位置。
- **典型模型**: GPT 系列 (Generative Pre-trained Transformer)
- **应用场景**: 文本生成、自动补全、对话系统等需要生成文本的任务。
- **工作方式**: 模型逐步生成文本, 每一步根据之前生成的内容预测下一个词。

3. Encoder-Decoder (编码器-解码器)

- **结构特点**: 包含编码器和解码器两个部分。编码器处理输入序列, 解码器生成输出序列, 解码器在生成每个词时, 既关注之前生成的词, 也关注编码器的输出。
- **典型模型**: 原始 Transformer、T5 (Text-to-Text Transfer Transformer)
- **应用场景**: 机器翻译、文本摘要、问答系统等需要将输入序列转换为输出序列的任务。
- **工作方式**: 编码器将输入序列编码为上下文表示, 解码器根据这些表示和已生成的词逐步生成输出序列。

PagedAttention Block 定义

$$K_j = (k_{(j-1)B+1}, \dots, k_{jB})$$

这表示第 j 个 Key block 包含:

- 第 $(j-1)B+1$ 到第 jB 个 token 的 Key 向量
- 一共刚好 B 个 key, 按顺序组合成一个 block

举个例子:

- Block size $B = 128$
- 那么第 1 个 block ($j = 1$) 包含:
 - Key 向量: $k_1, k_2, \dots, k_{128}$
 - Value 向量: $v_1, v_2, \dots, v_{128}$
- 第 2 个 block ($j = 2$) 包含:
 - Key 向量: $k_{129}, \dots, k_{256}$
 - Value 向量: $v_{129}, \dots, v_{256}$

PagedAttention 中基于 block 的注意力计算公式

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t / \sqrt{d})}$$

- A_{ij} 表示第 i 个 token (查询向量 q_i) 与第 j 个 KV block (键向量 K_j) 之间的注意力权重。
- q_i 为第 i 个 token 的查询向量 (Query vector)。
- K_j 为第 j 个 KV block 的键向量 (Key vector)。
- d 表示模型维度大小, 用于缩放注意力分数, 防止过大的数值导致梯度消失。
- 分母是归一化因子, 确保所有的注意力权重之和为 1。

$$o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top$$

- o_i 为第 i 个 token 的注意力输出向量 (Output vector) 。
- V_j 为第 j 个 KV block 的值向量 (Value vector) 。
- $A_{ij}^\top$ 表示注意力权重向量的转置，与值向量 V_j 相乘以加权求和，产生最终的输出。
- 每个 A_{ij} 实际上是一个行向量，包含了第 i 个 token 对于第 j 个 KV block 内所有 token 的注意力得分：

$$A_{ij} = (a_{i,(j-1)B+1}, \dots, a_{i,jB})$$

- 每个 KV block 的大小固定为 B ，这样通过分块计算注意力，可以显著降低内存使用，并提高计算效率。这就是 PagedAttention 机制的核心思想。

CUDA kernel

CUDA kernel 是一种运行在 GPU 上的并行函数，它通过启动大量线程并行执行相同的代码，从而实现高效的并行计算，是 CUDA 编程模型的核心概念。

参考资料

- How To Reduce LLM Decoding Time With KV-Caching: <https://newsletter.theaiedge.io/p/how-to-reduce-llm-decoding-time-with>
- Introduction to vLLM and PagedAttention: <https://blog.runpod.io/introduction-to-vllm-and-how-to-run-vllm-on-runpod-serverless/>
- The First vLLM Meetup: <https://docs.google.com/presentation/d/1QL-XPFXiFpDBh86DbEegFXBXFXjix4v032GhShbKf3s/edit?usp=sharing>
- 图解大模型计算加速系列之：vLLM核心技术PagedAttention原理: <https://mp.weixin.qq.com/s/5EniAmFf1v9RdxI5-CwiQ>
- The KV Cache: Memory Usage in Transformers: <https://www.youtube.com/watch?v=80blUggRjf4&t=319s>
- vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention: <https://blog.vllm.ai/2023/06/20/vllm.html>